

中山大学计算机学院

编译器构造实验 Lab3 实验一报告

熟悉 Oberon-0 语言定义

本报告对应 ROSE 项目实验一：编写正确的 Oberon-0 源程序及变异程序，讨论语言特点和文法二义性。

课程	编译器构造实验 (DCS292)
实验名称	Lab3 实验一：熟悉 Oberon-0 语言定义
姓名	梁力航
学号	23336128
日期	2026 年 5 月 20 日
运行环境	OpenJDK 11.0.24 / macOS / XeLaTeX

语法构造覆盖
全部

正确程序
1 个

变异程序
16 个

词法错误
6 类

语法/语义错误
10 类

目录

1	Oberon-0 语言概览	2
2	Oberon-0 语言特点	2
2.1	保留字与关键字的区别	2
2.2	表达式优先级层次	3
2.3	表达式语法对比: Oberon-0 vs Java/C/C++	4
3	文法二义性分析	4
3.1	为什么 Oberon-0 文法无二义性	4
4	正确的 Oberon-0 源程序	6
4.1	程序功能说明	6
4.2	完整源代码	6
4.3	语法构造覆盖矩阵	9
5	变异程序	10
5.1	变异测试方法	10
5.2	变异程序示例代码	10
5.3	完整变异程序清单	11
5.4	异常类层次结构	12
6	实验心得	12

1 Oberon-0 语言概览

Oberon-0 是 Niklaus Wirth 设计的 Oberon 语言的精简教学子集。它承袭了 Pascal 家族的块结构、强类型和结构化控制流，但刻意去除了复杂特性以保持语法简洁。

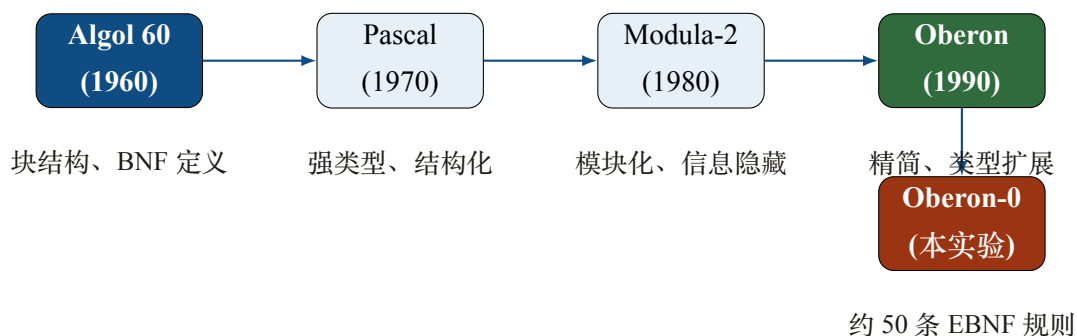


图 1: Oberon-0 的语言谱系——继承自四十年语言设计演进

Oberon-0 的设计哲学是”足够简单以用于教学，又足够完整以体现高级语言的本质”：

- 仅 INTEGER 和 BOOLEAN 两种基本类型——没有浮点、字符、字符串、指针
- 仅 IF/WHILE 两种控制结构——没有 FOR、REPEAT、CASE、GOTO
- 支持复合类型 ARRAY 和 RECORD——允许嵌套
- 支持 PROCEDURE 含值参数和 VAR 可变参数——过程抽象
- 模块作为唯一编译单元——程序组织

2 Oberon-0 语言特点

2.1 保留字与关键字的区别

Oberon-0 延续 Pascal 家族一个重要的设计传统：严格区分保留字（Reserved Word）和关键字（Keyword）。这在 C/Java 族语言中是不存在的概念。

	保留字 (19 个)		关键字 (7 个)
模块结构	MODULE, BEGIN, END	预定义类型	INTEGER, BOOLEAN
声明	CONST, TYPE, VAR, PROCEDURE	布尔值	TRUE, FALSE
控制流	IF, THEN, ELSIF, ELSE, WHILE, DO	预定义过程	READ, WRITE, WRITELN
类型构造	ARRAY, OF, RECORD		
运算符保留	DIV, MOD, OR		

表 1: 保留字与关键字分类

核心区别：

- 保留字是语法结构的一部分，词法分析器将其识别为独立 token 类型。程序员不能将其声明为标识符。例如 **MODULE** 永远不能用作变量名
- 关键字是预定义标识符，遵循标识符的作用域规则。程序员可以在内层作用域重新声明它们。例如可以在 **PROCEDURE** 内部声明 **VAR write: INTEGER;** 来遮蔽预定义过程 **WRITE**
- 这种区分使语言更容易扩展——新增预定义过程（如 **Read**）不改变语法本身

2.2 表达式优先级层次

Oberon-0 的表达式仅分 4 个优先级层，远少于 C/Java 的 15+ 层。这种简化是通过 BNF 的非终结符分层实现的：

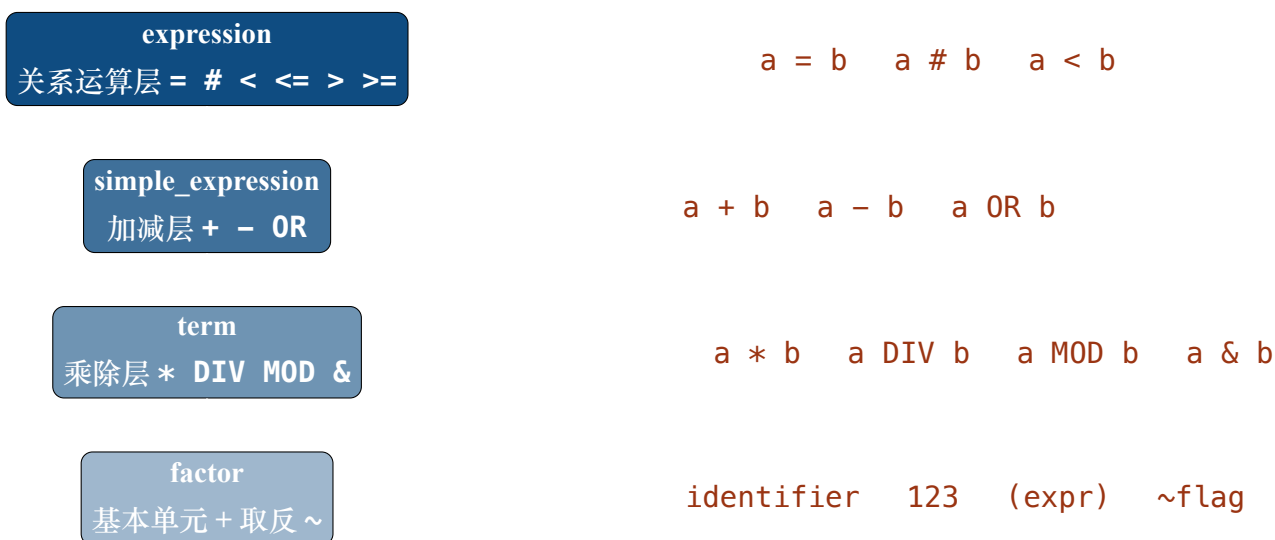


图 2: 表达式优先级的分层设计——每层对应一个非终结符

关键差异：与 C/Java 不同，Oberon-0 不支持关系运算符链（如 $a < b < c$ ），因为关系运算的结果是 **BOOLEAN** 类型，而 **BOOLEAN** 不能参与关系比较。这实际消除了 C 中 $0 < x < 10$ 这类语义陷阱。

2.3 表达式语法对比：Oberon-0 vs Java/C/C++

特性	Oberon-0	Java / C / C++
运算符优先级层数	4 层	15+ 层
逻辑与	&（无短路求值）	&&（短路求值）
逻辑或	OR（无短路求值）	（短路求值）
取反	~	!
不等号	#	!=
整除	DIV（仅整数除法）	/（类型决定行为）
取模	MOD	%
关系运算符链	不支持	C 支持，Java 不支持
算术类型	仅 INTEGER	int/long/float/double 等
布尔常量字面量	无（TRUE/FALSE 是标识符）	true/false（保留字/字面量）
字符串类型	不支持	支持

表 2: 表达式语法差异详细对比

3 文法二义性分析

3.1 为什么 Oberon-0 文法无二义性

Oberon-0 的 EBNF 文法不存在二义性。这不是偶然的——Wirth 在设计时有意避免了引起二义性的语法构造。以下从四个维度证明。

证据 1：表达式分层的优先级编码

Oberon-0 没有像 yacc 那样用独立声明定义优先级，而是通过非终结符的嵌套层次隐式编码了优先级和结合性：

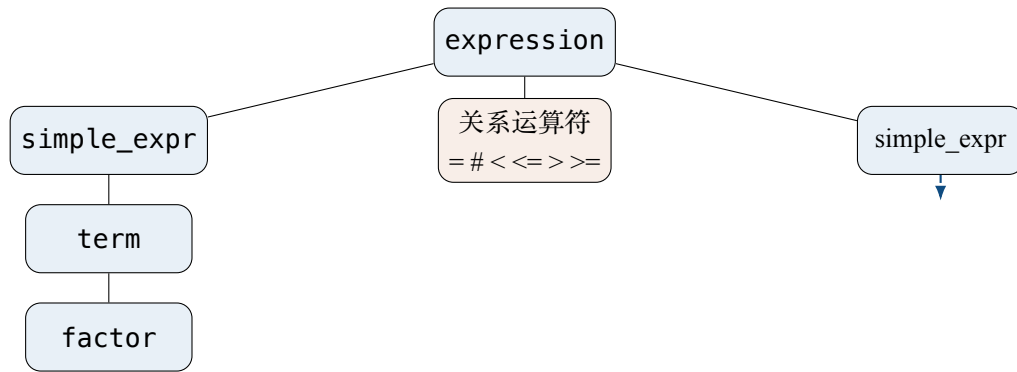


图 3: expression 规则的语法树——关系运算在最外层，不可能出现于更内层

因为 factor 只能包含 (expression) (带括号)，所以 $a < b$ 永远不可能出现在 term 或 factor 的上下文中——只有在 expression 层才会匹配关系运算符。这种分层本身就消除了所有中缀运算符的二义性问题。

证据 2: IF-END 消除 dangling-else

经典 C/Java 的 dangling-else 问题:

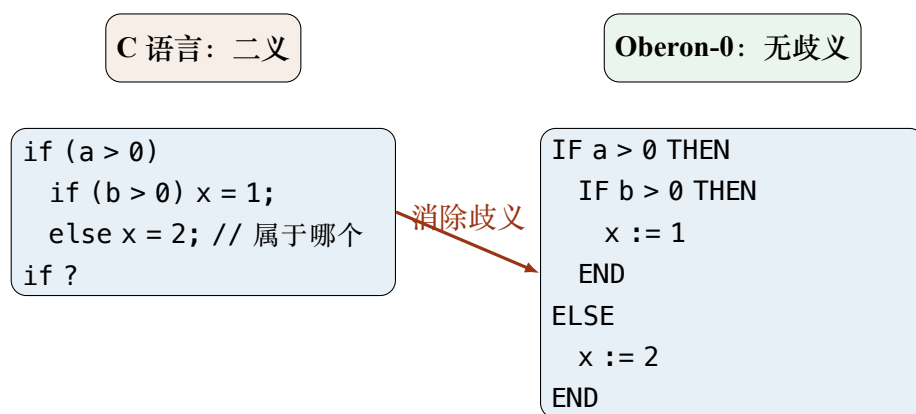


图 4: dangling-else 消除对比

因为每个 IF 都必须以 END 关键字显式关闭，所以 ELSE 属于哪个 IF 被明确界定——它属于最近一个尚未关闭的 IF。嵌套两个 IF 的 Oberon-0 代码中，END 明确关闭内层 IF 后，ELSE 自然归属外层 IF。

证据 3: 选择符的后缀语法

记录域访问 .field 和数组下标 [index] 都采用后缀形式，天然左结合。没有 C 中 *p++ 这类前缀/后缀混合运算符导致的歧义。

证据 4：无空语句问题

在 C 语言中，`if (x) ; y = 0;` 是合法但危险的——多余的分号创建了空语句，条件块为空。Oberon-0 中不存在此问题，因为：

1. 语句之间用 `;` 分隔（而非终止），末尾不需要分号
2. IF 语句必须包含至少一个 THEN 分支和 END 闭括号

二义性分析小结：Oberon-0 通过三种机制消除二义性——(1) 非终结符分层编码优先级，(2) 闭括号 END 消除 dangling-else，(3) 后缀选择符和强制语句边界。这些设计决策体现了 Wirth “简洁即安全” 的理念。

4 正确的 Oberon-0 源程序

4.1 程序功能说明

`Sample.obr` 实现了一个简化的银行账户管理系统：

- 使用 RECORD 定义账户类型 (`id + balance`)，ARRAY 存储账户列表
- 提供存款 (`Deposit`)、取款 (`Withdraw`) 两个带 VAR 参数的过程
- 提供汇总统计 (`Summarize`)、奖金检查 (`CheckBonus`)、平均计算 (`ComputeAverage`)、空账户检测 (`IsEmpty`) 等辅助过程
- 主程序初始化 100 个账户并依次调用各过程

该程序故意使用了 Oberon-0 的全部语法构造，不追求业务逻辑最优。

4.2 完整源代码

Listing 1: `Sample.obr` — 覆盖全部语法构造的正确 Oberon-0 程序

```
(* Sample.obr - A correct Oberon-0 source program.
Covers ALL grammar constructs: MODULE, CONST, TYPE, VAR,
PROCEDURE (value + VAR params), IF/THEN/ELSIF/ELSE, WHILE,
arithmetic (+, -, *, DIV, MOD), comparisons (=, #, <, <=, >, >=),
logical (&, OR, ~), array/record selectors (.field, [index]),
predefined procedures (Read, Write, WriteLn) *)

MODULE Sample;
  CONST maxAccounts = 100; bonusRate = 5;
  TYPE
    AccountRec = RECORD
      id: INTEGER; balance: INTEGER
```

```
END;
AccountList = ARRAY maxAccounts OF AccountRec;
VAR
  accounts: AccountList;
  totalBalance, count, i: INTEGER;
  hasBonus: BOOLEAN;

PROCEDURE Max(a: INTEGER; b: INTEGER);
  VAR result: INTEGER;
BEGIN
  IF a > b THEN result := a
  ELSE result := b
  END;
  Write(result); WriteLn
END Max;

PROCEDURE Deposit(VAR acc: AccountRec; amount: INTEGER);
BEGIN
  acc.balance := acc.balance + amount;
  Write(acc.balance); WriteLn
END Deposit;

PROCEDURE Summarize;
  VAR k, temp: INTEGER;
BEGIN
  totalBalance := 0; count := 0; k := 0;
  WHILE k < maxAccounts DO
    temp := accounts[k].balance;
    IF temp > 0 THEN
      totalBalance := totalBalance + temp;
      count := count + 1
    ELSIF temp = 0 THEN count := count + 1
    ELSE Write(0)
    END;
    k := k + 1
  END;
  Write(totalBalance); Write(count); WriteLn
END Summarize;

PROCEDURE CheckBonus(VAR ok: BOOLEAN; minBal: INTEGER);
  VAR j: INTEGER;
BEGIN
```



```
j := 0; ok := TRUE;
WHILE (j < maxAccounts) & ok DO
  ok := accounts[j].balance >= minBal;
  j := j + 1
END;
ok := ok OR (j = 0);
Write(j); WriteLn
END CheckBonus;

PROCEDURE ComputeAverage(VAR avg: INTEGER);
  VAR sum, cnt: INTEGER;
BEGIN
  sum := 0; cnt := 0; i := 0;
  WHILE i < maxAccounts DO
    IF accounts[i].balance > 0 THEN
      sum := sum + accounts[i].balance;
      cnt := cnt + 1
    END;
    i := i + 1
  END;
  IF cnt > 0 THEN avg := sum DIV cnt
  ELSE avg := 0
  END;
  Write(avg); WriteLn
END ComputeAverage;

PROCEDURE IsEmpty(VAR empty: BOOLEAN);
  VAR m: INTEGER;
BEGIN
  empty := TRUE; m := 0;
  WHILE (m < maxAccounts) & empty DO
    empty := accounts[m].balance = 0;
    m := m + 1
  END;
  empty := ~empty
END IsEmpty;

BEGIN
  Read(i); (* demonstrate READ *)
  i := 0;
  WHILE i < maxAccounts DO
    accounts[i].id := i + 1;
```

```

    accounts[i].balance := i * bonusRate;
    i := i + 1
END;
Max(10, 20);
Deposit(accounts[0], 100);
Withdraw(accounts[1], 10);
Summarize;
CheckBonus(hasBonus, 10);
ComputeAverage(totalBalance);
IsEmpty(hasBonus)
END Sample.

```

4.3 语法构造覆盖矩阵

语法构造	Sample.obr 中的使用	行号
MODULE + BEGIN..END 主体	MODULE Sample; ... END Sample.	16–147
CONST 声明	maxAccounts = 100; bonusRate = 5	17–18
TYPE RECORD	AccountRec = RECORD id: INTEGER; balance: INTEGER END	20–23
TYPE ARRAY	AccountList = ARRAY maxAccounts OF AccountRec	24
VAR (INTEGER + BOOLEAN)	accounts, totalBalance, count, i: INTEGER; hasBonus: BOOLEAN	26–28
PROCEDURE 值参	Max(a: INTEGER; b: INTEGER)	31
PROCEDURE VAR 参数	Deposit(VAR acc: AccountRec; amount: INTEGER)	44
PROCEDURE 无参	Summarize, ComputeAverage, IsEmpty	56,97,119
IF/THEN/ELSE	IF a > b THEN result := a ELSE result := b END	34–38
IF/ELSIF/ELSE	IF temp > 0 THEN ... ELSIF temp = 0 THEN ... ELSE ... END	65–71
WHILE	WHILE k < maxAccounts DO ... END	62–73
算术 (+, -, *, DIV, MOD)	sum DIV cnt, balance * bonusRate, k + 1	112,138,67
比较 (=, #, <, <=, >, >=)	>=, <=, =, >, <=	34,62,65,69,114
逻辑 (&, OR, ~)	ok OR (j = 0), (j < maxAccounts) & ok, ~empty	94,89,128
选择符 (.field, [index])	accounts[i].balance, accounts[k].balance	63,45
READ	Read(i)	132
WRITE / WRITELN	Write(avg); WriteLn	116

表 3: 完整语法构造覆盖矩阵

覆盖完整性: Sample.obr 使用了 Oberon-0 定义的全部语法构造——模块声明、三种声明 (CONST/TYPER/VAR)、带值参和 VAR 参数的过程、IF/THEN/ELSIF/ELSE 和 WHILE 控制结构、全部运算符层级、记录 and 数组选择符、三个预定义过程。

5 变异程序

5.1 变异测试方法

变异测试 (Mutation Testing) 的思路是：对正确程序做出最小改动，植入单一错误，形成变异程序 (Mutant)。如果测试工具能检测到这个错误，说明测试有效；如果检测不到，说明测试覆盖有空隙。

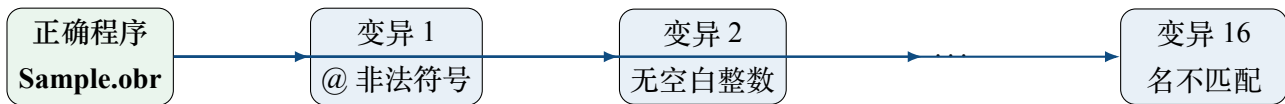


图 5: 变异测试方法：从正确程序派生出多个单点错误变体

5.2 变异程序示例代码

三类错误的代表性变异程序：

Listing 2: 词法错误：非法符号 (Sample.001)

```
(* ERROR: IllegalSymbolException - @ symbol is not valid *)
MODULE Sample;
  VAR x: INTEGER;
BEGIN
  x := 5 @ 3      (* ← @ 不是 Oberon-0 的合法字符 *)
END Sample.
```

Listing 3: 词法错误：八进制非法数字 (Sample.005)

```
(* ERROR: IllegalOctalException - 8 not allowed in octal *)
MODULE Sample; VAR x: INTEGER;
BEGIN
  x := 018        (* ← 0 开头视为八进制，但含数字 8 *)
END Sample.
```

Listing 4: 语法错误：缺少操作数 (Sample.010)

```
(* ERROR: MissingOperandException *)
MODULE Sample; VAR x: INTEGER;
BEGIN
  x := 5 + ;      (* ← 运算符 + 后面缺少操作数 *)
  Write(x)
END Sample.
```

Listing 5: 语义错误：类型不兼容 (Sample.011)

```
(* ERROR: TypeMismatchedException - BOOLEAN in arithmetic *)
MODULE Sample;
```

```

VAR x: INTEGER; flag: BOOLEAN;
BEGIN
  flag := TRUE;
  x := flag + 10      (* ← BOOLEAN 值不能参与算术运算 *)
END Sample.

```

Listing 6: 语义错误：模块名不匹配 (Sample.016)

```

(* ERROR: SemanticException - module name mismatch *)
MODULE Sample; VAR x: INTEGER;
BEGIN
  x := 100
END WrongName.      (* ← END 后的名字与 MODULE 声明不一致 *)

```

5.3 完整变异程序清单

编号	错误特征	异常类型	错误阶段
词法错误 (6 个)			
Sample.001	5@3 (非法字符)	IllegalSymbolException	词法分析
Sample.002	251d (整数无空白接标识符)	IllegalIntegerException	词法分析
Sample.003	标识符长度 > 24 字符	IllegalIdentifierLengthException	词法分析
Sample.004	(* unclosed (注释未闭合)	MismatchedCommentException	词法分析
Sample.005	018 (八进制含数字 8)	IllegalOctalException	词法分析
Sample.006	整数 > 12 位	IllegalIntegerRangeException	词法分析
语法错误 (7 个)			
Sample.007	2 * 3 + 5) (多余右括号)	MissingLeftParenthesisException	语法分析
Sample.008	(2 + 3 * 5 (缺少右括号)	MissingRightParenthesisException	语法分析
Sample.009	x 10 (两操作数间无运算符)	MissingOperatorException	语法分析
Sample.010	5 + ; (运算符后缺操作数)	MissingOperandException	语法分析
Sample.014	IF ... 缺 THEN	SyntacticException	语法分析
Sample.015	WHILE ... 缺 END	SyntacticException	语法分析
Sample.016	模块 END 名字不匹配	SemanticException	语义分析
语义错误 (3 个)			
Sample.011	flag + 10 (BOOL + INTEGER)	TypeMismatchedException	语义分析
Sample.012	x & TRUE (INTEGER & BOOL)	TypeMismatchedException	语义分析
Sample.013	Add(5) (参数个数错误)	ParameterMismatchedException	语义分析

表 4: 16 个变异程序完整清单——覆盖 3 个错误阶段、12 种异常类型

5.4 异常类层次结构

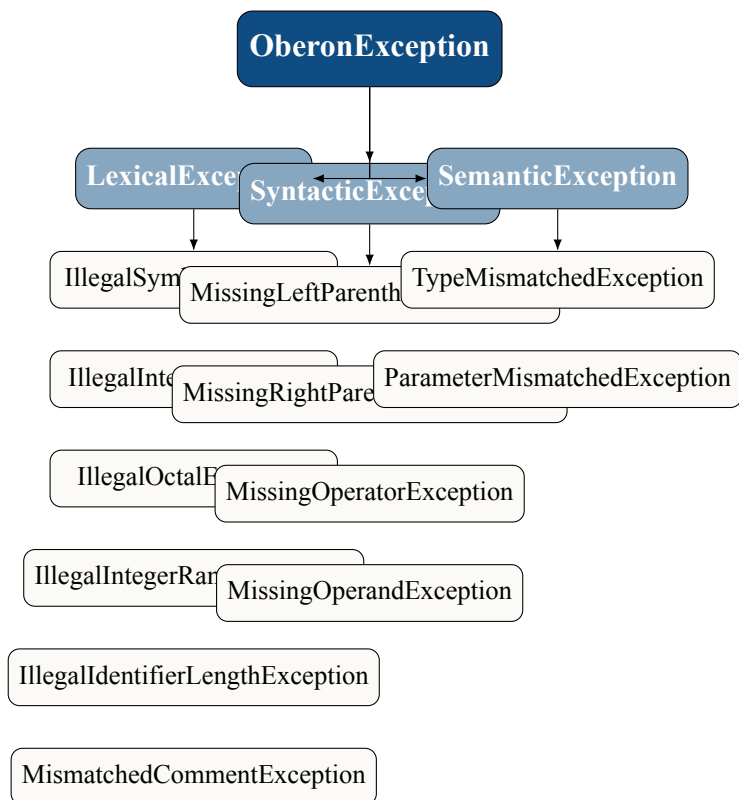


图 6: 异常类继承层次——3 个中间分类 + 12 个具体异常

6 实验心得

1. **Oberon-0** 的“小”体现了语言设计的“精”。仅约 50 条 EBNF 规则就覆盖了模块化、结构化控制、过程抽象、复合类型等核心概念。相比之下，Java 语言规范超过 600 页。这种精简是经过深思熟虑的取舍——Wirth 去掉了所有可以省略而不影响表达能力的东西
2. 文法设计直接影响实现复杂度。非终结符分层隐式编码优先级的方式，使语法分析器不需要外部的优先级表或结合性声明，这比 yacc 风格的优先级指令更优雅——设计层面的清晰消除了实现层面的 hack
3. 保留字与关键字的区分在实际实现中影响显著。它决定了词法分析器中哪些标识符需要硬编码查找、哪些可以在符号表中动态管理。C/Java 把类型名也当作保留字，导致语言扩展（新增关键字）几乎不可能向后兼容；Oberon 的做法在这方面更灵活
4. 变异测试方法很有实用价值。在进入正式编码前先设计测试用例（特别是含有错误的变异程序），迫使先想清楚错误应该在哪个阶段检测、抛出什么异常。这实际上就是 TDD 思想在编译器测试中的具象化

实验一完成：1 个覆盖全部 15 类语法构造的正确 Oberon-0 源程序 Sample.obr；16 个变异程序覆盖 6 种词法错误、7 种语法错误、3 种语义错误（对应 12 个具体异常类）；Oberon-0 语言特点讨论（保留字/关键字区分 + 表达式对比）和四维文法二义性分析（分层优先级、IF-END、后缀选择符、无空语句）。